

AI Research Project – Socratic Multimodal RAG Tutor for Occupational Therapy Education

Project Overview

The goal is to build a Socratic AI Tutor for Occupational Therapy (OT) education that helps students learn Anatomy and Neuroscience through guided questioning rather than direct answers.

This is not a chatbot project. The system will combine Retrieval-Augmented Generation (RAG), multimodal image understanding, conversation management, student memory, assessment, and evaluation into a complete end-to-end educational platform.

Core Requirements

1. Knowledge Base & Dataset Creation

Curate and prepare the dataset.

Sources include:

- OpenStax Anatomy & Physiology
- AnatomyTOOL
- MedPix
- Additional open-source OT/Anatomy educational resources

Deliverables include:

- Text corpus preparation
 - Chunking and embedding pipeline
 - 20–30 anatomical diagrams/images
 - Image metadata in JSON format
 - Ground-truth evaluation dataset for testing
-

2. Retrieval-Augmented Generation (RAG)

Implement a complete RAG pipeline using local vector storage.

Tech:

- ChromaDB
- FAISS

Requirements:

- Semantic retrieval
 - Context grounding
 - Citation/source tracking
 - Hallucination reduction
-

3. Socratic Tutoring Logic (Critical Requirement)

The tutor must follow a strict “Tutor-Not-Teller” methodology.

Requirements:

- Direct answers must NOT be revealed during the first two conversational turns.
- System must use progressive hints.
- System must guide students toward answers through questioning.
- Knowledge masking must be implemented.

Example:

Student: “What is the function of the cerebellum?”

The system should not immediately provide the answer and should instead guide the student through reasoning.

4. Conversation State Management

Implement an agent/controller that manages:

- Rapport-building phase
- Tutoring phase
- Assessment phase
- Mastery summary phase

The system must track:

- Turn count
 - Student progress
 - Tutoring state
 - Answer reveal eligibility
-

5. Multimodal Diagram Tutoring

The system must accept uploaded images.

Requirements:

- Identify anatomical structures from unseen images.
- Support blind testing.
- Work with images not used during demonstrations.
- **Generate Socratic follow-up questions based on detected structures.**

Tech:

- LLaVA
 - Other open-source vision-language models
-

6. Student Memory & Personalization

Implement session memory that tracks:

- Previous mistakes
- Weak knowledge areas
- Historical performance

The system should revisit previously missed concepts.

Example:

“Last session you struggled with Cranial Nerve VII. Let’s revisit it.”

7. Assessment Engine

The tutor must evaluate reasoning rather than multiple-choice answers.

Requirements:

- Clinical scenario generation
 - Student response evaluation
 - Comparison against reference material
 - Mastery scoring
 - Performance summaries
-

8. Student Dashboard

Create a dashboard displaying:

- Weak areas
- Performance trends

- Session history
 - Topic mastery
-

9. Accessibility Features

Implement at least one:

- Speech-to-Text
 - Text-to-Speech
-

Open-Source Requirement

The architecture must be optimized for local/open-source inference.

Preferred stack:

- Llama 3
- Mistral
- LLaVA
- Sentence Transformers
- ChromaDB
- FAISS

External APIs (OpenAI, Claude) may be used for comparison or benchmarking but the primary solution should remain reproducible without paid services.

Evaluation & Benchmarking

The project include benchmarking against a baseline system.

Examples:

Baseline:

- Standard QA chatbot

Proposed System:

- Socratic Tutor

Evaluation should include:

- RAGAS Faithfulness

- Answer Relevance
- Hallucination analysis
- Socratic compliance
- User-learning effectiveness

Required:

- Quantitative results
 - Qualitative analysis
 - Sample conversations
 - Graphs and visualizations
-

Blind Testing Requirement

The grading team will test the system with their own unseen prompts and images.

The application must:

- Accept arbitrary image uploads
 - Accept arbitrary user questions
 - Handle invalid inputs gracefully
 - Avoid crashes and traceback errors
 - Include proper exception handling
-

Deployment

A live, functioning deployment.

- Streamlit Community Cloud
 - Public URL
 - Fully operational interface
 - Secure credential management
 - No personal information exposed
-

Docker

- Dockerfile
 - Containerized deployment setup
 - Environment configuration
 - Setup documentation
-

Deliverables

Technical Report

ACL paper format (approximately 5 pages)

Including:

- Literature review
- System architecture
- Experimental setup
- Evaluation results
- Benchmark comparisons
- Observations and conclusions

Source Code

Complete GIT repository with documentation.

Presentation

PowerPoint presentation with demo content.

Video Demonstration

End-to-end walkthrough of the system.

Tech Stack

- Python
- Streamlit/FastAPI
- LangChain or LlamaIndex
- RAG systems
- ChromaDB / FAISS
- Llama 3 / Mistral
- LLaVA
- Docker
- Evaluation frameworks (RAGAS)
- Research-oriented AI projects